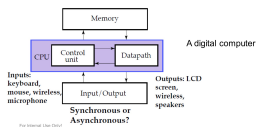


Chapter 1 Number System

1. Digital Systems



Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
0	0000	00	0x0
1	0001	01	0x1
2	0010	02	0x2
3	0011	03	0x3
4	0100	04	0x4
5	0101	05	0x5
6	0110	06	0x6
7	0111	07	0x7
8	1000	10	0x8
9	1001	11	0x9
10	1010	12	0xA
11	1011	13	0xB
12	1100	14	0xC
13	1101	15	0xD
14	1110	16	0xE
15	1111	17	0xF

2. Number Systems

① Radix-r to Decimal

Let r be the radix

$$D = d_{n-1}d_{n-2}\dots d_1d_0.d_{-1}d_{-2}\dots d_{-m}, 0 \leq d_i < r$$

↓ Most significant Digit Least significant Digit

$$\text{has value } D = d_{n-1} \cdot r^{n-1} + d_{n-2} \cdot r^{n-2} + \dots + d_1 \cdot r + d_0 + d_{-1} \cdot r^{-1} + \dots + d_{-m} \cdot r^{-m}$$

$$D = \sum_{i=-m}^{n-1} d_i r^i$$

② Decimal to Radix-r

e.g. $(13.625)_{10} \rightarrow \text{radix } 2$

(i) Integer part	Quotient	Remainder
13/2	6	1
6/2	3	0
3/2	1	1
1/2	0	1

$\Rightarrow (1101)_2 = (13)_{10}$

(ii) Fraction part	Integer	Fraction
0.625×2	1	.25
0.25×2	0	.5
0.5×2	1	.0

$\Rightarrow (0.101)_2 = (0.625)_{10}$

③ Radix-r to Radix-r

(i) Radix- r_1 to decimal to Radix- r_2

(ii) Binary related

e.g. Binary to Octal Binary to Hexadecimal

$$(1010110.010)_2 \quad (10010110.010)_2$$

$$(26.2)_8 \quad (16.4)_{16}$$

Octal to Hexadecimal

$$(26.2)_8 \quad \text{Octal} \rightarrow \text{Binary} \rightarrow \text{Hexadecimal}$$

$$(00010110.0100)_2$$

$$(16.4)_{16}$$

④ Common Notions

Bit 位 Byte 字节

1 Byte = 8 bits

⑤ Binary calculation

(i) Addition (same as decimal)

$$\begin{array}{r} 11101 \\ + 10111 \\ \hline 1010101 \end{array} \quad \begin{array}{l} = 61 \text{ augend} \\ = 23 \text{ addend} \\ = 84 \text{ sum} \end{array}$$

(2) Subtraction

$$\begin{array}{r} 1001101 \\ - 012021 \\ \hline 0110110 \end{array} \quad \begin{array}{l} = 77 \text{ minnend} \\ = 23 \text{ subtrahend} \\ = 54 \text{ diff} \end{array}$$

*借1当2 按位取借

⑥ Complement

(i) Find current radix r

(ii) $\begin{cases} (r-1)'s \text{ complement} & \text{反码} \\ r's \text{ complement} & \text{补码} \end{cases}$ inished radix complement Radix complement

有 n digits 的 r -radix 数 N

反码: $(r-1)'s \text{ complement} = (r^n - 1) - N$ (每一位为 $r-1-N$ 的第 n 位)

补码: $r's \text{ complement} = \text{反码} + 1$

* For binary system

反码: 1's complement: $0 \rightarrow 1, 1 \rightarrow 0$

补码: 2's complement: 反码+1 or non-0 LSB unchange, then $0 \rightarrow 1, 1 \rightarrow 0$

(iii) Subtraction with complement

两个有 n 个 digits 的 r 进制无符号数 $M-N$:

$$M + (N \text{ 的补码}) = M + r^n - N$$

$$M \geq N \text{ 非负} \rightarrow \text{结果} - r^n \rightarrow M - N$$

$$M < N \rightarrow \text{结果} - r^n = (N - M) \text{ 取反} \rightarrow M - N$$

(iv) Signed Binary Number

Signed 2's complement is Best 补码表示法

For positive is 0 it set

For negative is 1 it's 2's complement

3. Binary code

① BCD code

(i) Each bit (1-10) represented by binary numbers

$$\text{e.g. } 396 \rightarrow 0011 \ 1111 \ 0110$$

(ii) BCD Addition:

if sum ≤ 1001 (9) valid BCD code

if sum ≥ 1010 (10) invalid BCD code

$$\begin{array}{r} 4 \ 0100 \\ + 5 \ 0101 \\ \hline 9 \ 1001 \end{array} \quad \begin{array}{r} 4 \ 0100 \\ + 8 \ 1000 \\ \hline 12 \ 10010 \end{array} \quad \begin{array}{r} 8 \ 1000 \\ + 9 \ 1001 \\ \hline 17 \ 10011 \end{array}$$

$$\begin{array}{r} 184 \ 0001 \ 1000 \ 0100 \\ + 576 \ 0101 \ 0111 \ 0110 \\ \hline 760 \ 0110 \ 1111 \ 1010 \\ \hline 0111 \ 0110 \ 1100 \ 0000 \\ \hline 7 \ 6 \ 0 \end{array}$$

(iii) BCD subtraction

Take 10's complement of subtrahend add to minnend

$$\text{e.g. } 109 - 132 = -23$$

$$\begin{array}{r} 109 \ 0001 \ 0000 \ 1001 \\ + 868 \ 1000 \ 0110 \ 1000 \\ \hline 1001 \ 0110 \ 1001 \\ + \ 0110 \ 1100 \ 1000 \\ \hline 1001 \ 0111 \ 0111 \end{array} \quad \begin{array}{l} 977's \ 10's \text{ complement} \\ 9 \ 7 \ 7 = -23 \end{array}$$

② Gray Code: Minimum change code

$$0000 \rightarrow 0001 \rightarrow 0011 \rightarrow 0010 \rightarrow 0110 \rightarrow 0111 \rightarrow 0101 \rightarrow \dots$$

③ Error detecting code Parity bit

even parity odd parity

$$\text{ASCII } A = 1000001 \quad 01000001 \quad 11000001$$

在 ASCII 码左加一位 1/0, 使其有奇校验时则有奇个 1, 偶校验时则有偶个 1

Chapter 2 Boolean Function & Logic Gate

1. Boolean Algebra

① Logic gate: AND, OR, NOT
 $AB, A+B, A'$

② Boolean equation $F = x + y'z$

Truth table

Theorem	Dual	Name
1 $x + 0 = x$	$x + 1 = x$	Identity
2 $x + 1 = 1$	$x + 0 = 0$	Null Element
3 $x + x = x$	$x \cdot x = x$	Idempotency
4 $(x')' = x$		Involution
5 $x + x' = 1$	$x \cdot x' = 0$	Complements
Theorem	Dual	Name
6 $xy = yx$	$x + y = y + x$	Commutativity
7 $(xy)z = x(yz)$	$(x + y) + z = x + (y + z)$	Associativity
8 $x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$	Distributivity
9 $x + xy = x$	$x(x + y) = x$	Absorption
10 $xy + xy' = x$	$(x + y)(x + y') = x$	Combining
11 $(x + y)y = xy$	$xy + y = x + y$	Simplification
12 $xy + xz + yz = xy + xz$	$(x + y)(x + z)(y + z) = (x + y)(x + z)$	Consensus
13 $(x + y)' = x'y'$	$(xy)' = x' + y'$	DeMorgan's law

操作顺序: $() > ' > \cdot > +$

对偶性: Duality: replace $\cdot$ with $+$, 0 with 1 , still holds
 术语:

Literal: a variable and its complement

Product term: literals connected by $\cdot$

Sum term: literals connected by $+$

e.g. $A'B'C + ABC + BC'$ 8 literals, 3 product term
 $(A+B+C)(A'+C)$ 5 literals, 2 sum term

* Each Boolean Function has only 1 representation in truth table
 but various way in algebraic form

Boolean Function complement

$F \rightarrow F'$ ① 取反

② 取 complement of each literal

③ Canonical and Standard form

(i) Min terms and Max terms

Min term: an AND term consists all literals possible

Max term: an OR term consists all literals possible

* $M_i = m_i'$ each max term is the complement of a minterm

(ii) Canonical form

Sum of Minterms

Product of Maxterms

e.g. $F = xy + x'z$

SoM: $F = x'y'z + x'yz + xy'z + xyz$

$= m_1 + m_3 + m_6 + m_7$

$= \Sigma(1, 3, 6, 7)$

POM: $F = (x+y+z)(x+y'+z)(x'+y+z)(x'+y'+z)$

$= M_0 \cdot M_2 \cdot M_4 \cdot M_5$

$= \Pi(0, 2, 4, 5)$

Prove SoM = POM

e.g. $F = \Sigma(1, 3, 6, 7) = m_1 + m_3 + m_6 + m_7$ (SoM)

$F' = \Sigma(0, 2, 4, 5) = m_0 + m_2 + m_4 + m_5$

$F = (F')' = (m_0 + m_2 + m_4 + m_5)' = m_0' m_2' m_4' m_5'$

$= M_0 M_2 M_4 M_5 = \Pi(0, 2, 4, 5)$ (POM)

Example: Minterms and maxterms for three binary variables

x	y	z	Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Represent a Function in Canonical Form
 find Truth table

(ii) Standard form

Canonical form doesn't necessarily be least literals

Standard form / Product of sum
 Sum of Product

(iv) Standard Gate

Complement	$x \rightarrow x'$	NAND	$x \cdot y \rightarrow x' \uparrow y' = (xy)'$
transfer	$x \rightarrow x$	NOR	$x \cdot y \rightarrow x \downarrow y = (x+y)'$
AND	$x \cdot y \rightarrow xy$	XOR	$x \cdot y \rightarrow x \oplus y = (x'y + xy')$
OR	$x + y \rightarrow x + y$	XNOR	$x \cdot y \rightarrow (x \oplus y)' = x'y + xy'$

(v) Multiple inputs

AND, OR, XOR, XNOR are both Commutative and Associative

NAND, NOR commutative but not associative

Chapter 3 Gate Level Minimization

1. Map method minimization: minimum number of terms with least literals

① Karnaugh Map: is a visual way indicating all possible ways to express a function

(i) Two-variable

A	B	F
m_0	0	0
m_1	0	1
m_2	1	1
m_3	1	1

$\rightarrow$ A B 0 1
 0 0 1 1
 1 1 1 1
 A+B

(ii) Three-variable

e.g. $F = A'C + A'B + ABC + BC$

BC	00	01	11	10
m_0	0	1	1	1
m_1	0	1	1	1
m_2	1	1	1	1
m_3	1	1	1	1

$\rightarrow C + A'B$

(iii) Four-variable

e.g. $CD \oplus 01110$

CD	00	01	11	10
00	0	1	1	0
01	1	0	0	1
11	1	0	0	1
10	0	1	1	0

$\rightarrow C + A'B$

* SOP 找 1, OR 找 1
 POS 找 0, AND 找 0

(iv) Implicant 蕴含

Any product term implies the function

Prime implicant 质蕴含

将最大可能的相邻方格进行合并获得的乘积项

Essential Prime implicant 基本蕴含项: 只被圈过一次

(v) Product of Sum simplification

Simplify F' to SoM $\rightarrow F = (F')'$ POS

e.g. $F(A, B, C, D) = \Sigma(2, 3, 7, 10, 11, 15)$

AB	00	01	11	10
00	0	0	1	1
01	0	0	1	0
11	0	0	1	0
10	0	0	1	1

$F' = \Sigma(0, 1, 4, 5, 6, 8, 9, 12, 13, 14)$
 $= C' + BD'$
 $F = (F')' = C(B' + D)$

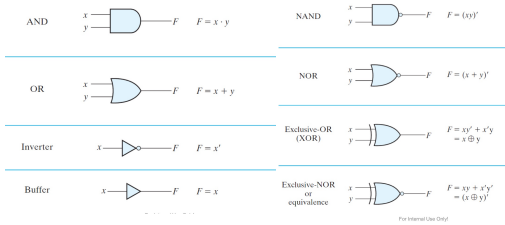
or: circle 0, eliminate changed item, save unchanged

if 1, complement; if 0, original $\Rightarrow C(B' + D)$

- (VI) Don't Care conditions
Put don't care item to X (can be 0/1)
In order to make circle larger, let X be 0 or 1
or make * circle less,

2. Two-level Implementation

Logic Gates



① NAND circuits 与非门

AND-invert, invert-OR



两级门电路实现:

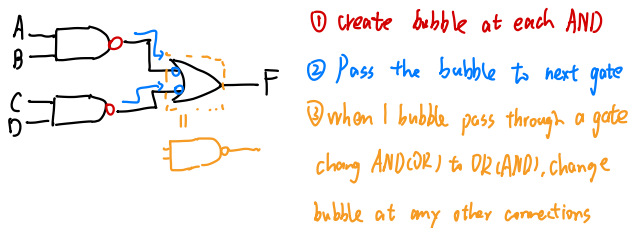
$$F(A, B, C, D) = AB + CD \quad (\text{start from SOP})$$

(i) Simplification

(ii) algebraic method (De Morgan's + double complement)

$$AB + CD = ((AB + CD)')' = ((AB)'(CD)')'$$

(iii) Bubble pushing method

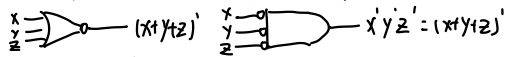


(iv) Multilevel NAND Implementation

- Change to and add • after
- Change to and add • before
- When no place to add •, change input to x', add inverter before F

② NOR Circuits 或非门

OR-invert, Invert-AND



两级门电路实现:

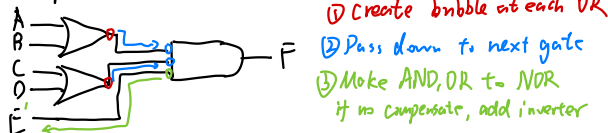
$$F(A, B, C, D, E) = (A+B)(C+D)E \quad (\text{start from POS})$$

(i) Simplification

(ii) algebraic method (De Morgan's + double complement)

$$(A+B)(C+D)E = (((A+B)(C+D)E)')' = ((A+B)' + (C+D)' + E')'$$

(iii) Graphic method

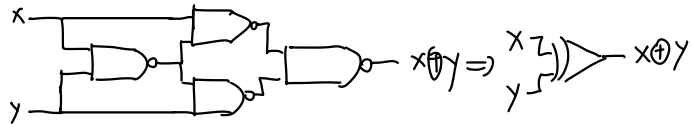


(iv) Multilevel NOR implementation

同上, OR, AND $\rightarrow$ NOR

$$\textcircled{3} \text{ XOR 异或 } x \oplus y = xy' + x'y$$

$$\text{XNOR 异同 } (x \oplus y)' = xy + x'y'$$



④ Odd Function

(i) XOR operation with 3+ variables can be

converted into an ordinary Boolean function

Replace $\oplus \rightarrow$ equivalent Boolean expression

$$A \oplus B \oplus C = (AB' + A'B)C + (AB' + A'B)C'$$

$$= \Sigma(1, 2, 4, 7)$$

(ii) Odd Function $\rightarrow$ odd * of variables = 1 $\Rightarrow F = 1$

(iii) Even Function $\rightarrow$ even * of variables = 1 $\Rightarrow F = 1$

(iv) Parity Generation and Checking

$$\text{Generate } P = x \oplus y \oplus z$$

$$C = x \oplus y \oplus z \oplus P = \begin{cases} 1 & \text{odd bit error} \\ 0 & \text{even bit error/correct} \end{cases}$$

⑤ Other Two-level Form

AND/NAND/OR/NOR have 16 combinations

- Eight are non-degenerate forms
- AND-OR $\Rightarrow$ standard sum-of-products
- NAND-NAND $\Rightarrow$ standard sum-of-products
- OR-AND $\Rightarrow$ standard product-of-sums
- NOR-NOR $\Rightarrow$ standard product-of-sums
- NAND-AND/AND-NOR $\Rightarrow$ AND-OR-INVERT (AOI)
 - complement of sum-of-products
- OR-NAND/NOR-OR $\Rightarrow$ OR-AND-INVERT (OAI)
 - complement of product-of-sums

NAND-AND = AND-NOR = AND-OR-Inverter 与或非

SOP required e.g. $F = (AB + CD + E)'$

AOI ($F' = \text{AND-OR}$, $F = \text{AND-NOR-Invert}$)

OR-AND = NOR-OR = OR-AND-Inverter 或非与

POS required e.g. $F = ((A+B)(C+D)E)'$

OAI ($F' = \text{OR-AND}$, $F = \text{OR-NAND-Invert}$)

Chapter 4 Combinational Logic

1. Logic Circuits for the Digital System

① Combinational Logic: output at current time based on present input combination

② Sequential circuits

Circuits that possess memory elements

2. Combinational Logic

① Analysis of Combinational Circuits

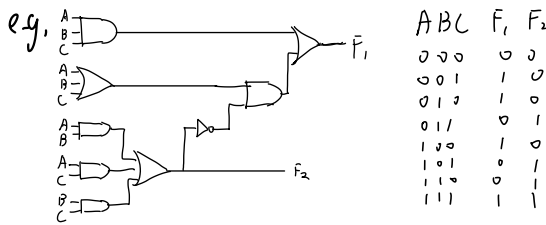
From given logic diagram

$\downarrow$

Develop Boolean function, truth table, explanation of circuits

(ii) Analysis with Truth Table

Set different variable 0/1 $\Rightarrow$ all possibilities



From truth table $\Rightarrow$

$F_1: A_0 \begin{matrix} 00 & 01 & 11 & 10 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{matrix}$
 $F_2: A_0 \begin{matrix} 00 & 01 & 11 & 10 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{matrix}$

$F_1 = AB'C + A'BL + A'B'C + ABC$
 $= A \oplus B \oplus C$

$F_2 = AB + AC + BC$

Functionality: Full adder: F_1 : sum, F_2 : carry

(ii) Analysis with Boolean Function Derivation

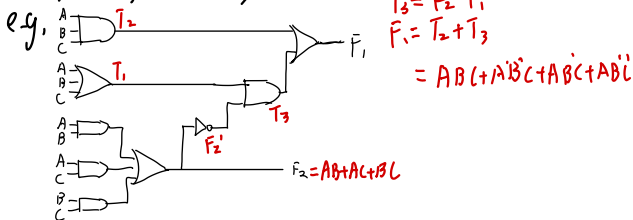
① Label all gate output that are function of only input no other intermediate

② Label all gates that are a function of inputs and the gates previously

③ List output Boolean Functions, squeeze intermediates.

④ Simplify if necessary

⑤ Explain if necessary



② Design of a combinational Circuits

(i) Specification: Find inputs, outputs, their symbols

(ii) Formulation: Derive truth table (functions)

(iii) Optimization: For each output, derive the simplified Boolean function

(iv) Logic diagram: using required technology mapping

3. Standard Components

① One-hot representation

Represent N elements with N bit

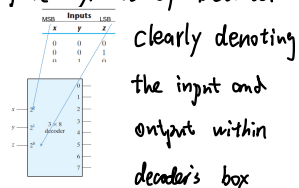
Exactly one bit is set

② Decoder 译码器

(i) Def: A combinational circuit converts binary from n inputs to $m \leq 2^n$ outputs: n -to- m line decoder

对 n 个输入变量产生 $\leq 2^n$ 个最小项

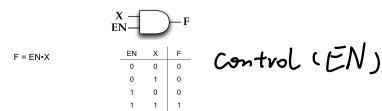
(ii) Graphic Symbol of Decoder



(iii) Main usages:

- Minterm generator
- Display decoding
- Data demultiplexing
- Address decoding

(iv) Enabling: permits an input signal to pass through a gate



③ Multiplexer 数据选择器 (MUX)

(i) Select binary information from 1 of (usually) 2^n inputs $\rightarrow$ 1 output

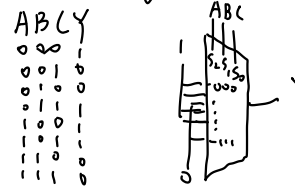
(ii) Composition:

Decoder + OR gates

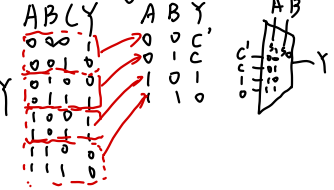
(iii) For Logic Implementation:

e.g. $Y(A, B, C) = \Sigma(0, 3, 4, 5)$

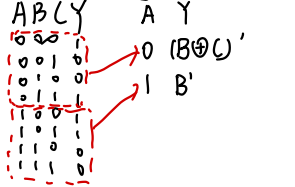
(i) Using 8:1 MUX



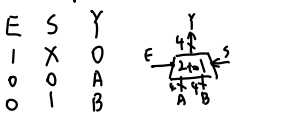
(ii) Using 4:1 MUX



(iii) Using 2:1 MUX

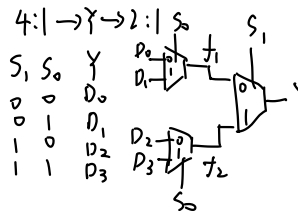


(iv) Quadruple 2:1 MUX (4-bit 2:1 MUX)



(v) MUX expansion

Wider MUX can be built with smaller multiplexers



④ Encoder 编码器 (Inverse of decoder)

From one-hot $\rightarrow$ binary inputs

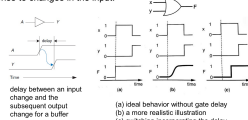
Illegal inputs $\rightarrow$ priority input (ensure only 1 input is encoded)

Assuming D_3 highest, D_0 lowest priority, X don't care, V valid output

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	x	x	0
1	0	0	0	0	0	1
x	1	0	0	0	1	1
x	x	1	0	1	0	1
x	x	x	1	1	1	1

⑤ Gate delays

When the input to a logic gate is changed, the output will not change immediately. The output of the gate experiences a propagation delay in response to changes in the input.



Ignoring delays can only analyse

its **steady-state behavior**

Chapter 5 Sequential Circuits

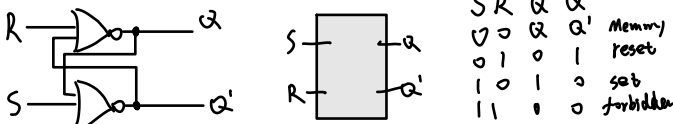
1. Categories

- ① Synchronous Circuits 同步电路: posedge 触发
- ② Asynchronous Circuits 异步电路: @* 触发

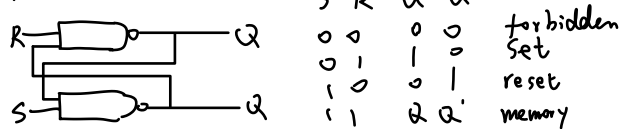
2. Storage elements

① Latch 锁存器: store 1 bit information

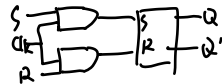
(i) SR latch



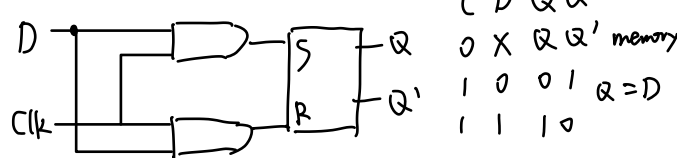
(ii) S' R' (low active)



(iii) clocked (en) SR latch

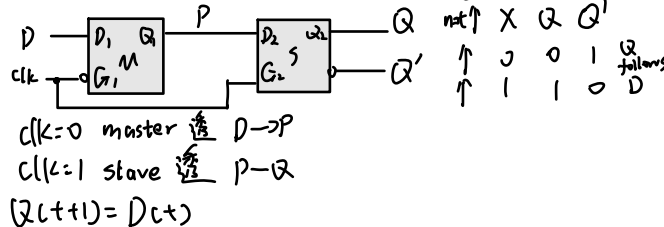


(iv) D latch



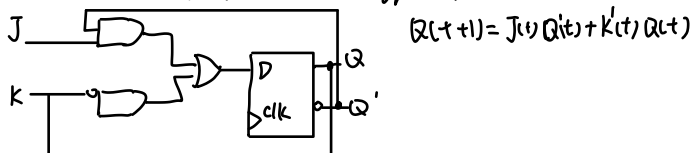
② Flip-flop

(i) D Flip-flop



(ii) JK Flip-flop

J=K=0, unchange. J=K=1, Q toggles, J=1, K=0, Q=1, J=0, K=1, Q=0



(iii) T-flipflop

T=0, keep, T=1, toggle

$Q(t+1) = T(t) \oplus Q(t)$

3. Analysis of clocked sequential circuit

(1) Derive input equation for FF inputs

(2) Derive states and output equations

Substitute input of FFs to FF's characteristic eqs.

Determine the output equations

(3) state, output table

(4) state diagram

4. Finite State machine:

(1) Mealy model: output from state and input

(2) Moore model: output from state only

5. State Minimization

(1) Def: m states, we need $\log_2 m$ FFs

(2) Steps: (i) Find rows in the state table have same output
(ii) If there is, remove 1, replace all b with a. (e.g. a, b)

6. Design of sequential circuits

(1) Need: description or timing diagram

(2) Formulation: state diagram

(3) generate state, output table

(4) minimize state if needed

(5) encode states

(6) state, output equations

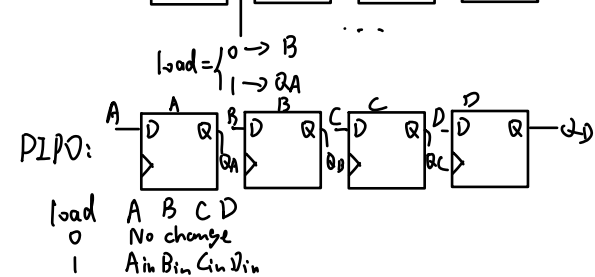
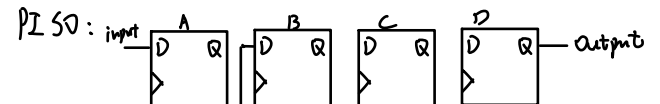
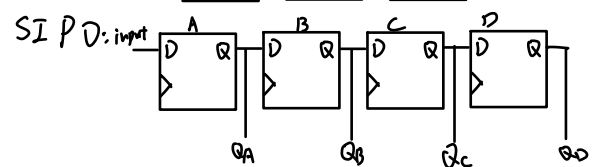
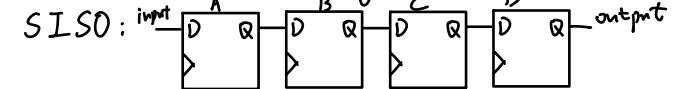
(7) choose FF, derive input eqs and output eqs.

D for data transfer, T for taking complement, J K versatile

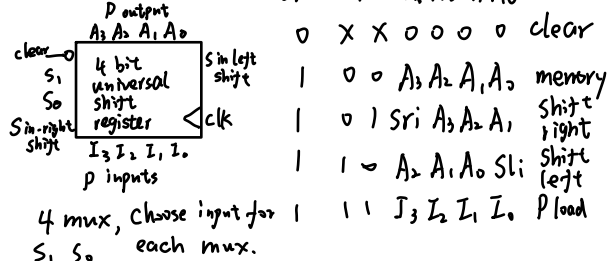
Chapter 6 Registers and Counters

1. Shift register: bitwise extension of FFs

(1) allow data move in the register



Universal:

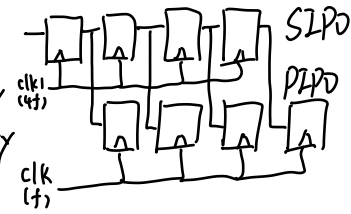


2. Serial/Parallel Transfer

Serial: slow, low complexity

Parallel: fast, high complexity

Clk for SLPD if faster!



3. Sequence generator

minimum FF of length $N \Rightarrow N \leq 2^n - 1$, n is # of FF

e.g. for 1001 (MSB first)

$z \quad Q_2 \quad Q_1 \quad Q_0 \quad z$

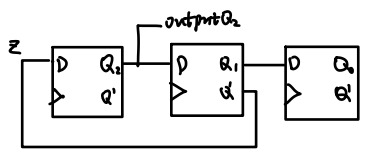
$x \quad 1 \quad 0 \quad 0 \quad 1$

$x \quad 0 \quad 1 \quad 0 \quad 1$

$x \quad 1 \quad 0 \quad 0 \quad 1$

$x \quad 1 \quad 0 \quad 0 \quad 1$

$\Rightarrow z = Q_1$



4. Asynchronous Counter

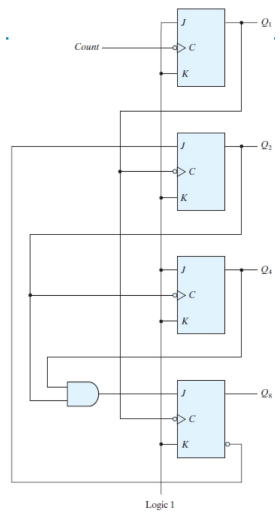
1) Ripple Counter

DPF: $A_0(t+1) = A_0'$, $A_1(t+1) = A_1(t)$, A_1 的clk是 A_0 取反...

TFF: $T_{in} = 1$, $A_{clk} = clk$, A_1 的clk是 A_0 取反...

BCD ripple counter:

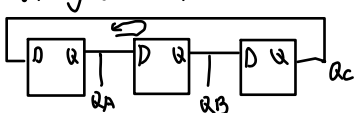
when QB from 1 to 0, trigger count for next higher decade.



5. Synchronous counter

Shift register counters

(i) Ring counter



(ii) Johnson counter switch tail

last FF complement output $\rightarrow$ input of first FF

(iii) Design

- ① design description or timing diagram
 - ② state diagram
 - ③ next state table
 - ④ choose FF
 - ⑤ derive FF inputs
 - ⑥ logic diagram
- N FF $\Rightarrow 2^N$ state
- note unused
- note don't care
- D: $Q(t+1) = D(t)$
- T: $Q(t+1) = T(t) \oplus Q(t)$
- JK: $Q(t+1) = J(t)Q'(t) + K(t)Q(t)$

Chapter 7 Arithmetic Circuits

1. Binary Adder

half adder: 2 bit addition

full adder: 3 bit addition

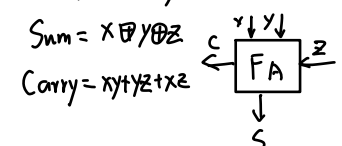
x, y, z (carry from lower bit)

$0+0=0$ $1+0=1$ $1+1=10$ Carry: xy

1011 augend

0001 addend

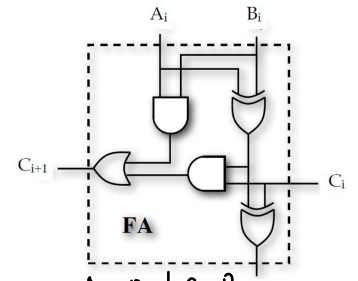
1100 sum



Ripple carry adder:

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + C_i (A_i \oplus B_i)$$



Carry lookahead adder

Carry-generate: $G_i = A_i B_i$

Carry-propagate: $P_i = A_i \oplus B_i$ ($P_i = A_i + B_i$)

$$C_{i+1} = G_i + P_i C_i = A_i B_i + (A_i \oplus B_i) C_i$$

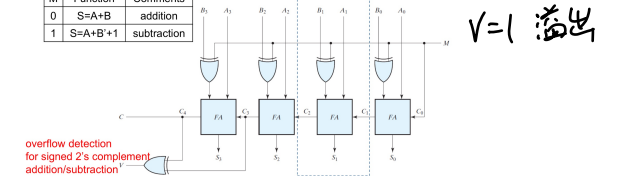
$$S_i = P_i \oplus C_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+2} = G_{i+1} + P_{i+1} C_{i+1} = G_{i+1} + P_{i+1} G_i + P_{i+1} P_i C_i \dots$$

2. Binary subtractor

Adding Minuend to 2's Complement of the subtrahend

M	Function	Comments
0	$S = A + B$	addition
1	$S = A + B + 1$	subtraction

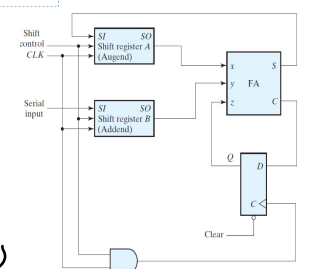


Serial adder

Reg A: Augend and sum

Reg B: Addend

LSB $\rightarrow$ MSB

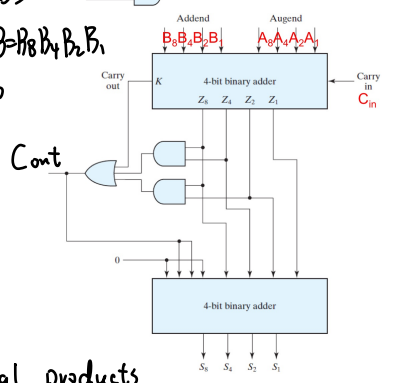


3. Decimal Adder (BCD)

$S = S_8 S_4 S_2 S_1$, $A = A_8 A_4 A_2 A_1$, $B = B_8 B_4 B_2 B_1$

$C = 1$ $\{k=1 \Rightarrow \text{add } 0110\}$

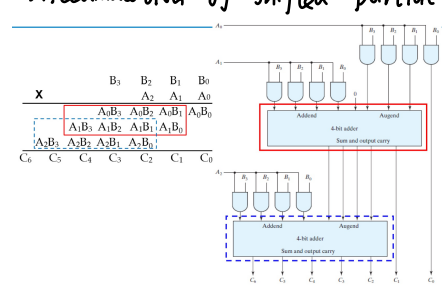
$A+B > 9$



4. Binary Multiplier

(1) Generation of partial products

(2) Accumulation of shifted partial products



5. Other arithmetic functions

Design by contraction

when 1 part is known $\Rightarrow$ simplify

Zero extension unsigned

$0110101 \rightarrow 16'b \quad 000000000110101$

Sign extension

$1110101 \rightarrow 16'b \quad 111111111110101$

8's complement